PL SQL

Lab1

Name: T Manikhanta Srikanth

Roll No: AM.SC.U4AIE23158

Q1

```
-- --q1
create or replace function check_number (a int) returns varchar as
$$
    declare
        b varchar;
    begin
        if a % 2 =0 then
            b := 'Even';
            return b;
        else
            b := 'Odd';
            return b;
        end if;
    end;
$$
language 'plpgsql';

select check_number(5)
```

Data Output Messages Notifications

Showing r

	check_number character varying
1	Odd

```
create function mysum(a int, b int) returns int as
$$
    begin
        return a + b;
    end
$$
language 'plpgsql';

select mysum(2,1)
```

Data Output Messages Notifications

	mysum integer
1	3

```sql
-- --q3
create table emp(eno varchar(10) primary key,
                 ename varchar(30),
                 sal numeric(8,2)
                );
insert into emp values('e1','Pavan',100000);

create function salary(in x emp.eno%type, out y emp.sal%type) as
$$
    begin
        select sal into y from emp where eno = x;
    end
$$
language 'plpgsql';

select salary('e1')
```

Data Output Messages Notifications

SQL Showi

	salary numeric
1	100000.00

```sql
-- --q4
alter table emp add column dno varchar(20);
create or replace function no_of_emp(x emp.dno%type) returns int as
$$
    declare
        empn int;
    begin
        select count(eno) into empn from emp group by eno having dno = x;
        return empn;
    end
$$
language 'plpgsql';
insert into emp values('e2','Pavan',100000,'forest');
select no_of_emp('forest');
```

Data Output Messages Notifications

SQL Showing rows: 1

	no_of_emp integer
1	1

```sql
CREATE TABLE department(dno varchar(20) primary key,
                 dname varchar(30),
                 loc varchar(50)
                 );
CREATE TABLE product (
    pr_no       INT PRIMARY KEY,
    pr_name     VARCHAR(50),
    unit_price  NUMERIC(10,2)
);

CREATE TABLE emp_sales (sl_no     INT PRIMARY KEY,
    emp_no    INT,
    pr_no     INT,
    qty_sold  INT,
    CONSTRAINT fk_product
        FOREIGN KEY (pr_no)
        REFERENCES product(pr_no)
);
```

```sql
INSERT INTO product VALUES
(1, 'Keyboard', 750.00),
(2, 'Mouse', 350.00),
(3, 'USB Cable', 150.00);
INSERT INTO emp_sales VALUES
(1, 101, 1, 400),
(2, 101, 2, 300),
(3, 101, 3, 500),

(4, 102, 1, 200),
(5, 102, 2, 150);
```

```sql
CREATE OR REPLACE FUNCTION sales_eligibility(empno INT)
RETURNS INT AS
$$
DECLARE
    total_qty INT;
BEGIN
    SELECT SUM(qty_sold)
    INTO total_qty
    FROM emp_sales
    WHERE emp_no = empno;

    IF total_qty IS NULL THEN
        RETURN 0; -- No sales
    END IF;

    IF total_qty >= 1000 THEN
        RETURN total_qty;
    ELSE
        RETURN 0;
    END IF;
END;
$$ LANGUAGE plpgsql;

SELECT sales_eligibility(101);
```

Data Output Messages Notifications

	sales_eligibility integer
1	1200

2.

(a)

```sql
CREATE OR REPLACE FUNCTION projectload(p_eno INT)
RETURNS NUMERIC AS
$$
DECLARE
    total_hours NUMERIC;
BEGIN
    SELECT COALESCE(SUM(prjt_hrs),0)
    INTO total_hours
    FROM emp_proj
    WHERE eno = p_eno;

    RETURN total_hours;
END;
$$ LANGUAGE plpgsql;

SELECT projectload(101);
```

Data Output Messages Notifications

	projectload numeric
1	30.00

(b)

```sql
--b
CREATE OR REPLACE FUNCTION update_salary_to_average()
RETURNS INT AS
$$
DECLARE
    avg_sal NUMERIC;
    updated_count INT;
BEGIN
    SELECT AVG(sal) INTO avg_sal FROM emp;

    UPDATE emp
    SET sal = avg_sal
    WHERE sal < avg_sal;

    GET DIAGNOSTICS updated_count = ROW_COUNT;

    RETURN updated_count;
END;
$$ LANGUAGE plpgsql;
SELECT update_salary_to_average();
```

Data Output Messages Notifications

SQL

	update_salary_to_average integer
1	1

3.

```sql
CREATE OR REPLACE FUNCTION delete_item_if_less_transactions(p_ino INT)
RETURNS TEXT AS
$$
DECLARE
    tx_count INT;
BEGIN
    SELECT COUNT(*) INTO tx_count
    FROM transaction_tbl
    WHERE ino = p_ino;

    IF tx_count < 2 THEN

        DELETE FROM transaction_tbl
        WHERE ino = p_ino;

        DELETE FROM item
        WHERE ino = p_ino;

        RETURN 'Item deleted (including its transactions)';
    ELSE
        RETURN 'Item NOT deleted (transactions >= 2)';
    END IF;
END;
$$ LANGUAGE plpgsql;

SELECT delete_item_if_less_transactions(2);
```

Data Output Messages Notifications

SQL Showing ro

	delete_item_if_less_transactions text
1	Item deleted (including its transactions)

4.

```sql
--a
CREATE OR REPLACE FUNCTION update_type(p_threshold NUMERIC, p_cno INT)
RETURNS TEXT AS
$$
DECLARE
    total NUMERIC;
    newtype CHAR;
BEGIN
    SELECT SUM(bal) INTO total
    FROM accounts
    WHERE cust_no = p_cno;

    IF total > p_threshold THEN
        newtype := 'A';
    ELSE
        newtype := 'B';
    END IF;

    UPDATE customer SET c_type = newtype WHERE cno = p_cno;

    RETURN newtype;
END;
$$ LANGUAGE plpgsql;
SELECT update_type(8000,1);
```

Data Output Messages Notifications

SQL Showing r

	update_type text
1	A

```sql
--b
CREATE OR REPLACE FUNCTION closebranch(p_close INT, p_take INT)
RETURNS TEXT AS
$$
BEGIN
    UPDATE accounts
    SET br_no = p_take
    WHERE br_no = p_close;

    DELETE FROM branches WHERE br_no = p_close;

    RETURN 'Branch closed and accounts transferred';
END;
$$ LANGUAGE plpgsql;
SELECT closebranch(10,20);
```

Data Output Messages Notifications

SQL Sho

	closebranch text
1	Branch closed and accounts transferred

```sql
CREATE OR REPLACE FUNCTION safe_withdraw(p_ac INT, p_amt NUMERIC)
RETURNS NUMERIC AS
$$
DECLARE
    cur_bal NUMERIC;
BEGIN
    SELECT bal INTO cur_bal FROM accounts WHERE ac_no = p_ac;

    IF cur_bal >= p_amt THEN
        UPDATE accounts
        SET bal = bal - p_amt
        WHERE ac_no = p_ac;

        RETURN cur_bal - p_amt;
    ELSE
        RETURN NULL;
    END IF;
END;
$$ LANGUAGE plpgsql;
SELECT safe_withdraw(200,3000);
```

Data Output Messages Notifications

SQL Showing

	safe_withdraw numeric
1	[null]